# INSTITUT UNIVERSITAIRE DES SCIENCES

(**IUS**)

## Faculté des Sciences et Technologies

(FST)

## TD Nº 1 – Modélisation

**Nom & Prénom :**

BICHOTTE Kendy

**Niveau** :

L3

**Date** : Mars 2026

## Introduction

La cryptographie constitue l'un des fondements essentiels de la sécurité informatique moderne. Elle permet de protéger l'intégrité, la confidentialité et l'authenticité des informations échangées dans un monde où les données circulent en permanence. Ces Travaux Dirigés proposent une exploration progressive de trois mécanismes cryptographiques fondamentaux : la génération de clés aléatoires, le chiffrement par décalage (César) et le chiffrement XOR. L'objectif est de développer une compréhension concrète des notions de sécurité, d'aléa, de robustesse des clés et de fonctionnement des systèmes symétriques simples.

## TD1 : Génération de clés aléatoires

## Objectif du TD : Comprendre les ensembles de caractères et l'entropie.

TD1.py × TD2.py undefined

C: > Users > afjte > Downloads > Modelisation > TD1.py > ...

```python
import string
import random
import math

def generate_key(charset, length):
    return ''.join(random.choice(charset) for _ in range(length))

def calculate_entropy(length, alphabet):
    L = len(alphabet)
    return length * math.log2(L)

# Définition des ensembles
char_sets = {
    "digits": string.digits,
    "ascii_lowercase": string.ascii_lowercase,
    "ascii_letters": string.ascii_letters,
    "complet (letters+digits+punct)": string.ascii_letters + string.digits + string.punctuation
}

# Impression du tableau
print(f"{'Ensemble':30} | {'Long.':5} | {'Clé Générée':35} | {'Entropie (bits)'}")
print("-"*90)

for name, chars in char_sets.items():
    for length in [8, 16, 32]:
        key = generate_key(chars, length)
        entropy = calculate_entropy(length, chars)
        print(f"{name:30} | {length:<5} | {key:<35} | {entropy: .2f}")
```

## Résultat attendu

```
PS C:\Users\afjte\Downloads\Modelisation>  & 'c:\Users\afjte\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\afjte\.vs
code\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63838' '--' 'c:\Users\afjte\Downloads\Modelisa
tion\TD1.py'
Ensemble                       | Long. | Clé Générée                        | Entropie (bits)
------------------------------------------------------------------------------------------
digits                         | 8     | 92200309                           |  26.58
digits                         | 16    | 0132023861327070                   |  53.15
digits                         | 32    | 41811244091735990505500702883647   |  106.30
ascii_lowercase                | 8     | lefqmaiu                           |  37.60
ascii_lowercase                | 16    | euegojprywpzebwm                   |  75.21
ascii_lowercase                | 32    | rodwrziulsuxyialjtcdredcwzvxyzol   |  150.41
ascii_letters                  | 8     | eazyCSyv                           |  45.60
ascii_letters                  | 16    | rrqsjPsPDyuaCWnL                   |  91.21
ascii_letters                  | 32    | EwHnBUrIQRDsKJfYKhXgxNsRUVBFNyIP   |  182.41
complet (letters+digits+punct) | 8     | `SGu?&`K                           |  52.44
complet (letters+digits+punct) | 16    | iQF%01H-yow)ez(o                   |  104.87
complet (letters+digits+punct) | 32    | v={0q2/h3.&!<2~Uw<8r}2qa~L6+*9*W   |  209.75
PS C:\Users\afjte\Downloads\Modelisation>
```

Plus la clé est longue, plus elle est difficile à deviner.

Plus l'alphabet est riche, plus chaque caractère apporte de bits d'information.

En d'autres termes, je peux dire que les clés courtes sont faibles, même avec un grand alphabet ; tandis que les clés longues avec alphabet varié sont très sécurisées. Cela implique qu'il est d'une importance capitale pourque les standards modernes comme AES, RSA pour ne citer que ceux-là exigent des clés de grande taille et diversité.

## TD2 : Chiffrement César

## Objectif du TD : Comprendre un chiffrement symétrique simple et la rotation des caractères.

```python
TD2.py > caesar_cipher
def caesar_cipher(message, shift):
    """
    Chiffre un message avec le chiffrement de César.
    """
    resultat = ""
    for char in message:
        if char.islower():  # lettres minuscules
            resultat += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
        elif char.isupper():  # lettres majuscules
            resultat += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
        elif char.isdigit():  # chiffres
            resultat += chr((ord(char) - ord('0') + shift) % 10 + ord('0'))
        else:  # autres caractères inchangés
            resultat += char
    return resultat


def caesar_decipher(message_chiffre, shift):
    """
    Déchiffrer revient à chiffrer avec un décalage négatif.
    """
    return caesar_cipher(message_chiffre, -shift)


# --- Tests demandés ---
tests = [
    ("abcxyz", 3),     # Minuscules
    ("ABCXYZ", 3),     # Majuscules
    ("012789", 3),     # Chiffres
    ("Code 123!", 5)   # Mélange
]

print(f"{'Original':<15} | {'Décalage':<8} | {'Chiffré':<15} | {'Déchiffré':<15}")
print("-" * 60)
for msg, s in tests:
    crypte = caesar_cipher(msg, s)
    decrypte = caesar_decipher(crypte, s)
    print(f"{msg:<15} | {s:<8} | {crypte:<15} | {decrypte:<15}")
```

## Résultat attendu

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS        Python Debug Console

wnloads\Modelisation\TD2.py'
Original        | Décalage | Chiffré         | Déchiffré
------------------------------------------------------------
abcxyz          | 3        | defabc          | abcxyz
ABCXYZ          | 3        | DEFABC          | ABCXYZ
012789          | 3        | 345012          | 012789
Code 123!       | 5        | Htij 678!       | Code 123!
PS C:\Users\afjte\Downloads\Modelisation>
```

Le résultat montre que :

- Le chiffrement de César est **symétrique** : ce qui est chiffré avec un décalage +n se retrouve avec un décalage −n.
- Les alphabets et chiffres sont traités séparément, chacun avec leur propre cycle (26 lettres, 10 chiffres).
- Les caractères non alphanumériques ne sont pas modifiés.

## TD3 : Chiffrement XOR simple

## Objectif du TD : Comprendre le chiffrement par clé avec opérations bit à bit.

```python
import random, string, math

def generer_cle(taille):
    """ Génère une clé aléatoire de longueur 'taille' """
    return ''.join(random.choice(string.printable) for _ in range(taille))

def xor_cipher(message, key):
    """ Chiffre ou déchiffre un message avec XOR """
    return ''.join(chr(ord(a) ^ ord(b)) for a, b in zip(message, key))

def calculer_entropie(cle):
    """ Calcule l'entropie de la clé : L * log2(R) """
    R = len(string.printable)  # environ 100 caractères
    L = len(cle)
    return L * math.log2(R)

# --- Exemple d'utilisation ---
message_original = "Kendy Bichotte 2026"
cle_aleatoire = generer_cle(len(message_original))

# Chiffrement
message_chiffre = xor_cipher(message_original, cle_aleatoire)

# Déchiffrement
message_final = xor_cipher(message_chiffre, cle_aleatoire)

# --- Affichage des résultats ---
print(f"Message original : {message_original}")
print(f"Clé (aléatoire) : {cle_aleatoire}")
print(f"Message chiffré : {repr(message_chiffre)}")
print(f"Message déchiffré : {message_final}")
print("Identique à l'original :", message_original == message_final)
print("Entropie de la clé :", calculer_entropie(cle_aleatoire))
```

## Résultat

```
PS C:\Users\afjte\Downloads\Modelisation>  & 'c:\Users\afjte\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\afjte\.vscode\extensions\
ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62182' '--' 'c:\Users\afjte\Downloads\Modelisation\TD3.py'
Message original : Kendy Bichotte 2026
Clé (aléatoire) : k&EcOs5&%"qo.OG6L9d
Message chiffré : ' C+\x076SwOFJ\x1e\x1bZ*g\x04|>R'
Message déchiffré : Kendy Bichotte 2026
Identique à l'original : True
Entropie de la clé : 126.23326760571976
PS C:\Users\afjte\Downloads\Modelisation>
```

Message original : « Kendy Bichotte 2026 »

Le chiffrement fonctionne, la clé est forte en théorie, mais la sécurité réelle dépend de la gestion de la clé (ne pas la réutiliser, ne pas la divulguer). Le principe du chiffrement XOR est simple mais réversible et l'entropie donne un indicateur de sécurité.

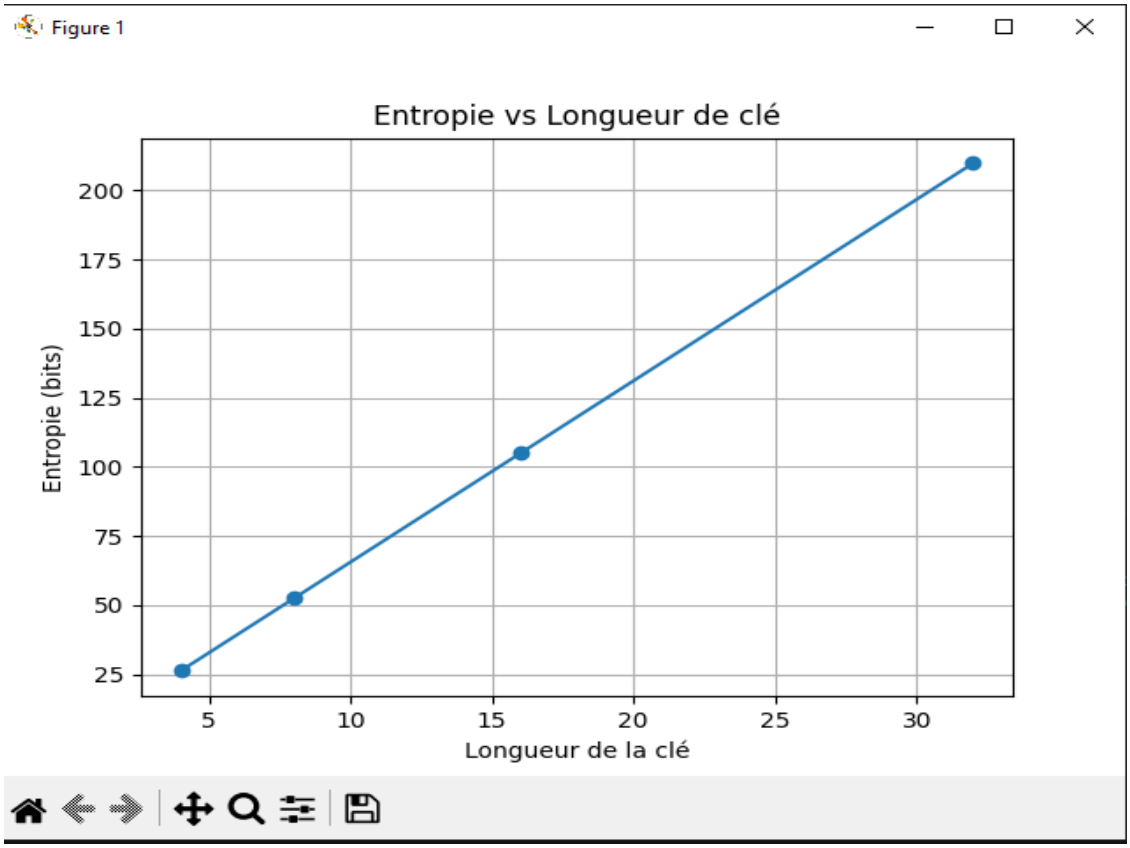
# TD4 : Génération et visualisation de l'entropie

## Objectif : Relier entropie, longueur de clé et alphabet.

TD4.py > ...

```python
import string
import random
import math
import matplotlib.pyplot as plt

def generate_key(charset, length):
    return ''.join(random.choice(charset) for _ in range(length))

def calculate_entropy(length, alphabet):
    L = len(alphabet)
    return length * math.log2(L)

lengths = [4, 8, 16, 32]
alphabet = string.ascii_letters + string.digits + string.punctuation
entropies = [calculate_entropy(l, alphabet) for l in lengths]

plt.plot(lengths, entropies, marker='o')
plt.title("Entropie vs Longueur de clé")
plt.xlabel("Longueur de la clé")
plt.ylabel("Entropie (bits)")
plt.grid(True)
plt.show()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console




# Conclusion

Ce Travaux Dirigés a permis d'explorer trois mécanismes fondamentaux de la sécurité informatique : la génération de clés aléatoires, le chiffrement par décalage (César) et le chiffrement XOR. Chacun de ces exercices a mis en lumière des notions essentielles telles que l'entropie, la robustesse d'une clé, la manipulation des caractères et l'importance des opérations bit à bit.

Génération de clés et entropie : La comparaison des clés générées avec différents ensembles de caractères montre clairement que plus l'alphabet est large, plus l'entropie augmente, et donc plus la clé est difficile à deviner.

Ce TD a ainsi illustré que la sécurité d'un système dépend autant de la longueur de la clé que de la richesse de l'alphabet utilisé.

Le chiffrement César, bien que simple à implémenter, démontre les limites des systèmes à faible entropie et à structure prévisible.

Le chiffrement XOR met en évidence un principe fondamental : un même opérateur peut servir à chiffrer et déchiffrer, ce qui simplifie la logique tout en offrant une sécurité correcte si — et seulement si — la clé est : totalement aléatoire, de même longueur que le message, utilisée une seule fois.